SEMINÁRIO 3 DE 3 · DESENVOLVIMENTO WEB

Angular

Arquitetura opinativa, TypeScript, injeção de dependências e Signals

Construindo o Mural de Eventos do Curso

Marcelly Almeida · Gabriel Barbosa · Marcelle Luna

ROTEIRO

Como vão ser as próximas 2 horas

Abertura	Fundamentos	Live coding	Oficina prática	Fechamento
10 min	30 min	20 min · RF1 e RF2	45 min · vocês programam RF3, RF4 e RF5	15 min
Marcelly	Gabriel		Marcelle (todos apoiam as duplas na oficina)	

O mesmo app de React e Vue — o que muda é **a forma de resolver**, não o problema.

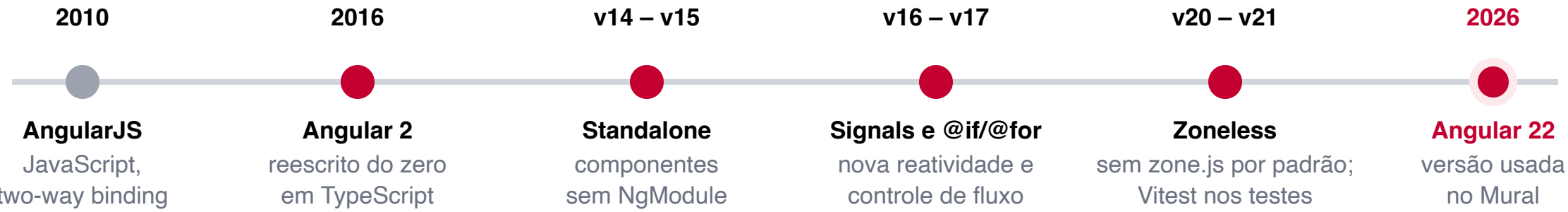
BLOCO 1 · MARCELLY

Abertura

De onde vem o Angular, qual é a filosofia dele e quando ele faz sentido.

CONTEXTO HISTÓRICO

De AngularJS ao Angular 22



Mantido pelo **Google** · uma versão major a cada ~6 meses · migrações guiadas pela CLI (`ng update`)

FILOSOFIA

"Baterias incluídas": uma plataforma, não só uma biblioteca



Por que isso importa

- Todo projeto Angular tem **a mesma estrutura**
- Quem entra na equipe já sabe onde está cada coisa
- Não é preciso escolher e combinar bibliotecas de terceiros

Quem usa

- Produtos do próprio Google, como o Google Cloud Console
- Sistemas corporativos grandes e de longa duração: bancos, governo, ERPs

FICHA COMPARATIVA

Ficha técnica do Angular

ITEM	ANGULAR
Natureza	**Framework completo.** Roteador, HTTP, formulários, testes e CLI são oficiais; só a API simulada (json-server) é externa.
Templates	HTML com bindings (`{{ }}` , `[ ]` , `( )`) e controle de fluxo `@if` , `@for` , `@switch` .
Reatividade	**Signals:** o Angular sabe quais valores cada template lê e atualiza só o que mudou. Sem zone.js (zoneless) desde a v21.
Estado local e global	Local: `signal` no componente. Global: **serviço singleton** (`providedIn: 'root'`) com signals.
Rotas e formulários	Oficiais: `@angular/router` e `@angular/forms` (Reactive Forms).
Criação de projeto	`ng new` — gera `src/app` com `app.config.ts` , `app.routes.ts` e o componente raiz.
Curva de aprendizado	**A mais alta das três:** TypeScript, DI, RxJS e muitas convenções desde o início.
Quando escolher	Equipes grandes, projetos longos, sistemas corporativos que precisam de padrão.

BLOCO 2 · MARCELLY E GABRIEL

Fundamentos

CLI · TypeScript · Componentes · **Injeção de dependências** · Signals · HTTP · Rotas · Formulários

ANGULAR CLI

A CLI cria e padroniza o projeto

TERMINAL

```
# cria o projeto (CSS, sem SSR)
ng new mural-eventos-angular

# gera arquivos já no padrão
ng generate interface models/course-event
ng generate service services/event
ng generate component event-list

# servidor de desenvolvimento
ng serve
```

ESTRUTURA GERADA

```
src/app/
├── app.config.ts    providers
├── app.routes.ts    rotas
├── app.ts           componente raiz
├── app.html / app.css / app.spec.ts
└── event-list/
    ├── event-list.ts
    ├── event-list.html
    ├── event-list.css
    └── event-list.spec.ts
```

Todo componente tem os mesmos 4 arquivos, no mesmo lugar. Em equipe grande, ninguém precisa "descobrir" a estrutura.

TYPESCRIPT

O compilador encontra o erro antes de executar

MODELO DO EVENTO

```
export interface CourseEvent {
  id: string;
  titulo: string;
  descricao?: string;
  data: string;
  local: string;
  categoria: EventCategory;
}
```

GENÉRICO NO HTTPCLIENT

```
list(): Observable<CourseEvent[]> {
  return this.http.get<CourseEvent[]>(url);
}
```

NO EDITOR

```
event.titlo
      ~~~~~
Property 'titlo' does not exist on type
'CourseEvent'. Did you mean 'titulo'?
```

COMPONENTES STANDALONE

Classe + template, ligados pelo decorator

```
@Component({
  selector: 'app-event-card',
  imports: [RouterLink, DatePipe],
  templateUrl: './event-card.html',
})
export class EventCard {
  event = input.required<CourseEvent>();
}
```

TRÊS FORMAS DE LIGAÇÃO NO TEMPLATE

Interpolação	`{{ event().titulo }}`
Property binding	`[routerLink]="['/eventos', id]"`
Event binding	`(click)="toggle(id)"`

Standalone: cada componente declara no `imports` o que usa — sem `NgModule`.

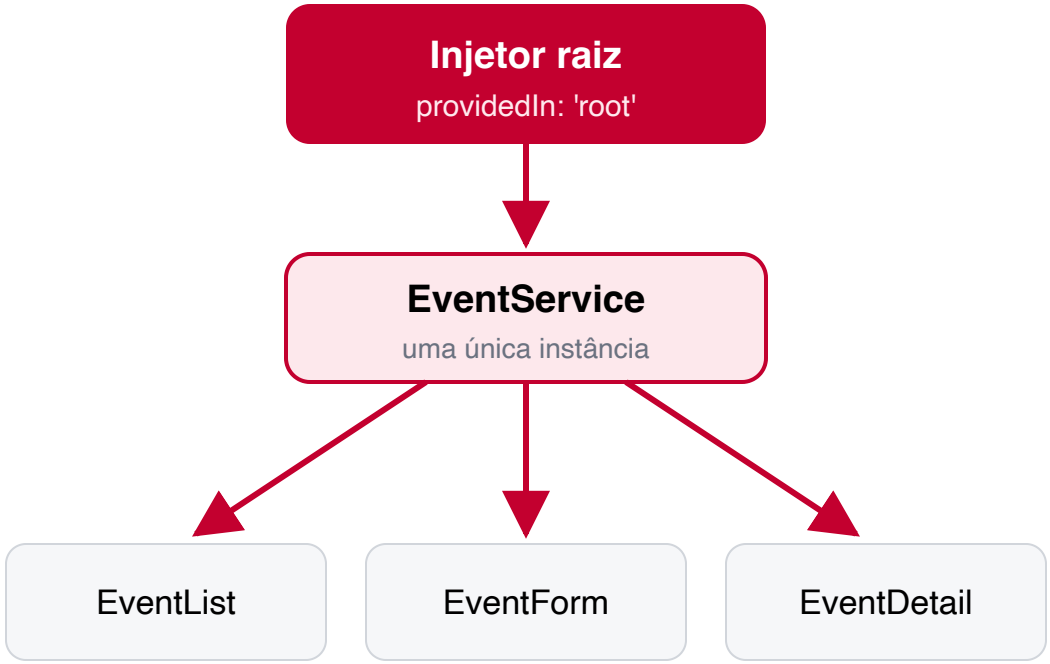
TEMPLATES

Controle de fluxo: @if, @for e @empty

```
@if (loading()) {
  <p>Carregando eventos…</p>
} @else if (loadError()) {
  <p>Não foi possível carregar.</p>
} @else {
  @for (event of filteredEvents();
        track event.id) {
    <app-event-card [event]="event" />
  } @empty {
    <p>Nenhum evento encontrado</p>
  }
}
```

- `track` é **obrigatório**: diz ao Angular como reconhecer cada item e reaproveitar o DOM
- `@empty` trata a lista vazia sem outro `@if`
- Projetos antigos usam `*ngIf` e `*ngFor` — você vai encontrá-los, mas a sintaxe nova é a recomendada

O componente pede; o injetor entrega



cada componente chama inject(EventService)

```
@Injectable({ providedIn: 'root' })
export class EventService {
  private http = inject(HttpClient);
}

export class EventList {
  private service = inject(EventService);
}
```

- Por que não `new EventService()`? O componente teria que saber **criar** o serviço e as dependências dele
- Novidade da v22: `@Service()`, um atalho para serviços já registrados no root

Trocar o serviço real por um dublê, sem mudar o componente

EVENT-LIST.SPEC.TS

```
const fake = {
  list: () => of([
    { id: '1', titulo: 'Git', ... },
  ]),
};

TestBed.configureTestingModule({
  imports: [EventList],
  providers: [
    { provide: EventService, useValue: fake },
  ],
});
```

LIGAÇÃO COM PROGRAMAÇÃO ORIENTADA A OBJETOS

Baixo acoplamento

O `EventList` não sabe como os dados chegam.

Inversão de dependência

O componente depende de um contrato; quem decide a implementação é o injetor.

Substituição em testes

Dados fixos no teste, API real em produção — o mesmo componente.

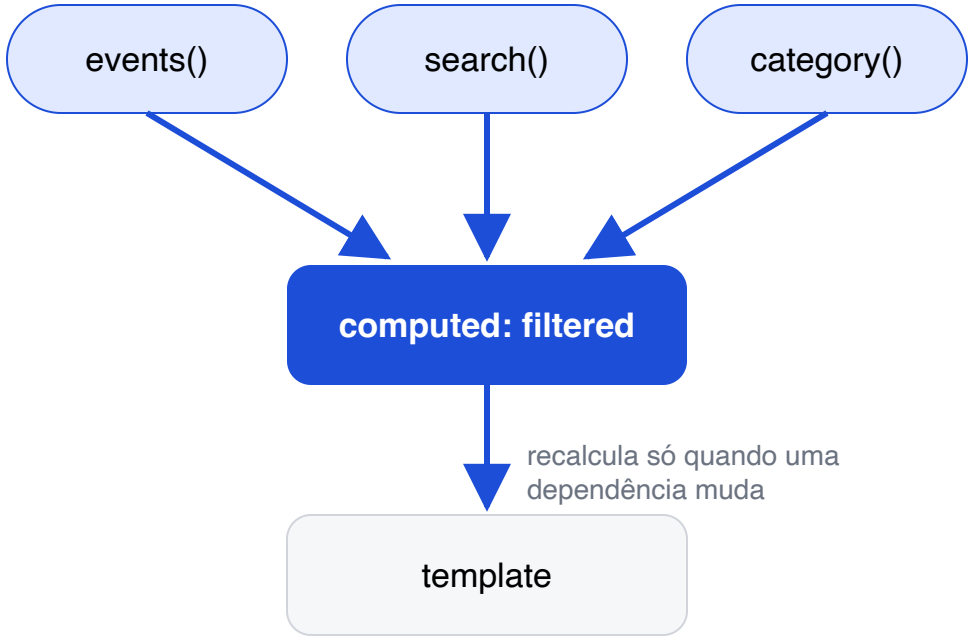
SIGNALS

Valores que avisam quando mudam

```
const search = signal('');
const category = signal('');

const filtered = computed(() =>
  events().filter((e) =>
    (!category() || e.categoria === category()) &&
    e.titulo.toLowerCase()
      .includes(search().toLowerCase())
  )
);

search.set('git');           // escrita
search.update((s) => s.trim()); // usa o atual
effect(() => console.log(search()));
```



SIGNALS · COMPARAÇÃO

Signals do Angular e ref/computed do Vue

	VUE (SEMINÁRIO ANTERIOR)	ANGULAR
Estado	`const busca = ref('')`	`search = signal('')`
Leitura	`busca.value`	`search()`
Escrita	`busca.value = 'git'`	`search.set('git')` ou `update(fn)`
Derivado	`computed(() => ...)`	`computed(() => ...)`
Efeito	`watchEffect(...)`	`effect(...)`

input() — pai → filho

`event = input.required<CourseEvent>()`

o card recebe o evento da lista

output() — filho → pai

`saved = output<CourseEvent>()`

o filho avisa o pai com `saved.emit(e)`

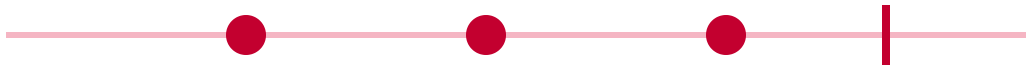
Por que um Observable e não uma Promise?

Promise



começa ao ser criada · 1 valor · não cancela

Observable



só começa no subscribe · N valores · cancelável · operadores RxJS

APP.CONFIG.TS

```
providers: [provideRouter(routes), provideHttpClient()]
```

FORMA 1 — SUBSCRIBE

```
this.service.list().subscribe({
  next: (e) => this.events.set(e),
  error: () => this.loadError.set(true),
});
```

FORMA 2 — CONVERTER EM SIGNAL

```
events = toSignal(this.service.list(),
  { initialValue: [] });
```

RxJS: biblioteca de fluxos assíncronos (map, filter, switchMap…) — usada pelo Angular por baixo.

ROTEAMENTO

Rotas, links e parâmetros

APP.ROUTES.TS

```ts
export const routes: Routes = [
  { path: '', component: EventList },
  { path: 'eventos/novo', component: EventForm },
  { path: 'eventos/:id', component: EventDetail },
  { path: '**', redirectTo: '' },
];
```

TEMPLATE

```html
<a [routerLink]="['/eventos', event.id]">...</a>
<router-outlet />
```

PARÂMETRO COMO INPUT DO COMPONENTE

```ts
provideRouter(routes,
  withComponentInputBinding())

export class EventDetail {
  id = input.required<string>(); // :id da URL
}
```

- `routerLink` troca a tela **sem recarregar** a página
- Alternativa clássica: `inject(ActivatedRoute)`

REACTIVE FORMS

O formulário é um objeto no TypeScript

```
form = new FormGroup({
  titulo: new FormControl('', [
    Validators.required, Validators.minLength(5)]),
  data: new FormControl('', [
    Validators.required, futureDateValidator]),
});

function futureDateValidator(c) {
  if (!c.value) return null;
  return c.value > today() ? null
                           : { pastDate: true };
}
```

ESTADOS DO CONTROLE

touched	o usuário saiu do campo
dirty	o valor foi alterado
invalid	algum validador retornou erro

COMPARAÇÃO

Vue	`v-model` liga campo e estado
React	componente controlado: `value` + `onChange`
Angular	modelo + validação ficam na classe; o template só se conecta

BLOCO 3 · GABRIEL · 20 MIN

Live coding: RF1 e RF2

1. Mostrar a estrutura gerada pela CLI
2. Criar a interface `CourseEvent` e registrar `provideHttpClient()`
3. Implementar o `EventService` com `list()`
4. Criar o `EventCard` com `input.required()` e o `EventList` consumindo o serviço
5. Busca e filtro com `signal` e `computed`; `@for` com `track` e `@empty`

RF1 — Listar

Cards ordenados por data, "Carregando eventos…" e mensagem de erro se a API cair.

RF2 — Filtrar e buscar

Categoria + título, juntos, a cada tecla — **sem nova requisição**.

Branch `live-coding` = ponto de partida · branch `start` = resultado esperado

BLOCO 4 · MARCELLE · 45 MIN · EM DUPLAS

Oficina prática: agora é com vocês

PREPARAÇÃO

```
git clone <url-do-repositório>
cd mural-eventos-angular
git checkout start
npm install
npm run api      # terminal 1
npm start        # terminal 2
```

Roteiro completo: `docs/roteiro-oficina.md`

CHECKPOINT	O QUÊ	TEMPO
1 · RF3	Cadastro com Reactive Forms e validador de data futura	18 min
2 · RF4	Detalhe com rota `eventos/:id` e 404	12 min
3 · RF5	Interesses com serviço + signal e contador no cabeçalho	15 min

Travou? `git checkout checkpoint-N` e continue do próximo.

Repositório: `https://github.com/Projeto-Angular-Web-UFMS/mural-eventos-angular`

BLOCO 5 · MARCELLE · 15 MIN

Fechamento

Quiz · pergunta-chave · reflexões · comparação com React e Vue

QUIZ · 5 MIN

5 perguntas sobre conceitos

1
Injeção de dependências

2
providedIn: 'root'

3
signal e computed

4
Observable × Promise

5
Validação em formulários

Acesse pelo link ou QR code que vamos mostrar agora

Nenhuma pergunta é de sintaxe: todas partem do Mural que vocês acabaram de construir.

PERGUNTA-CHAVE

Que problema a injeção de dependências resolve que importar diretamente uma classe não resolve?

VERSÃO A

```
export class EventList {
  private service = new EventService(
    new HttpClient(/* ...? */),
  );
}
```

VERSÃO B

```
export class EventList {
  private service = inject(EventService);
}
```

Qual é mais fácil de **testar**? E de trocar a API simulada por uma real — ou por dados fixos?

REFLEXÃO

Observables ou Signals? Os dois, cada um no seu papel

Observable — fluxo no tempo

- Requisições HTTP, eventos, WebSockets
- Valores que **chegam** ao longo do tempo
- Cancelar, combinar, repetir (RxJS)

Signal — estado atual

- O que a tela mostra **agora**
- Leitura síncrona no template: `events()`
- Derivados com `computed`, sem gerenciar inscrições



REFLEXÃO · POSTURA CRÍTICA

O custo das convenções

O que custa

- **Mais arquivos e mais código** que React e Vue para o mesmo Mural
- **Curva de aprendizado alta:** TypeScript, DI, RxJS e decorators desde o primeiro dia
- **Mudanças rápidas:** standalone, Signals, @if, zoneless, @Service — muito material antigo na internet
- **Menos liberdade:** o framework decide a estrutura por você

Quando compensa

- **Equipes grandes:** todo mundo escreve código parecido
- **Projetos longos:** quem chega daqui a 3 anos encontra a mesma estrutura
- **Sistemas corporativos:** formulários complexos, muitas telas, testes
- **Atualizações guiadas:** `ng update` migra o código entre versões

> Para um protótipo de fim de semana, o Angular é pesado. Para um sistema que 20 pessoas vão manter por anos, as convenções se pagam.

COMPARAÇÃO

O mesmo Mural, três soluções

	REACT	VUE	ANGULAR
Natureza	Biblioteca de UI	Framework progressivo	Plataforma completa
Interface	JSX	SFC com diretivas (`v-if`)	Template HTML com `@if` / `@for`
Estado	`useState`	`ref` / `computed`	`signal` / `computed`
Estado global (RF5)	Context ou biblioteca externa	Pinia ou composable	Serviço singleton com DI
Rotas e formulários	Comunidade	Vue Router oficial; forms com `v-model`	Oficiais (Router e Reactive Forms)
TypeScript	Opcional	Opcional	**Obrigatório**
Ideia central	UI como função do estado	Reatividade automática	Convenção + injeção de dependências

OBRIGADO!

Dúvidas?

Material completo no repositório

- `README.md` — instalação e branches
- `docs/roteiro-oficina.md` — os três checkpoints
- `docs/quiz.md` — questões com gabarito e justificativas